fn sample_geometric_buffer

Vicki Xu (@matchaginseng), Hanwen Zhang, Zachary Ratliff (@zachratliff)

May 2, 2026

This document proves soundness of `sample_geometric_buffer` in `mod.rs` at commit f5bb719 (outdated[1]).

1 Hoare Triple

Precondition

Compiler-verified

None

Caller-verified

None

Pseudocode

```
def sample_geometric_buffer(
    buffer_len: usize, constant_time: bool
) -> Optional[uint]:  #
    if constant_time:
        buf = bytearray(buffer_len)
        fill_bytes(buf)  # mutates in-place
        ret = None
        for i in range(buffer_len):
            # find first nonzero event
            if buf[i] > 0:
                # compute index of first nonzero bit buffer
                cand = 8 * i + buf[i].leading_zeroes()  #
                ret = cand if ret is None else min(ret, cand)
        return ret
    else:
        for i in range(buffer_len):
            buf = bytearray(1)
            fill_bytes(buf)  # mutates in-place
            if buf[0] > 0:
                return 8 * i + buf[0].leading_zeroes()

        return None
```

Postcondition

Theorem 1.1. For any setting of the input arguments, `sample_geometric_buffer` either raises an exception if there is insufficient system entropy, or returns `sample` where `sample` is drawn from a discrete distribution.

[1] See new changes with `git diff f5bb719..30b2e21 rust/src/traits/samplers/geometric/mod.rs`

`sample` is either `geo` where `geo` is a sample from the $Geometric(p = 0.5)$ distribution, and is less than $buffer_len * 8$, or `None` with probability $2^{-buffer_len*8}$.

Proof. `sample_geometric_buffer` uses `fill_bytes` as a subroutine to generate a buffer of `buffer_len` bytes. For each bit b in the buffer it follows that $\Pr[b = 1] = \frac{1}{2}$ and $\Pr[b = 0] = \frac{1}{2}$. If there is some bit in the buffer equal to 1, the position of the *first* such bit is a zero-indexed draw from the Geometric distribution $Geom(p)$ with $p = 0.5$, by definition of a Geometric random variable. If the buffer is zero, the function returns `None`. □